

Odniesienia do wymagań (wszystkie wykresy pochodzą z Clear ML):

1. Należy przygotować sobie bazę danych do klasyfikacji zdjęć (binarną lub wieloklasową).

### Baza danych: Owoce.zip + Gruszka.zip

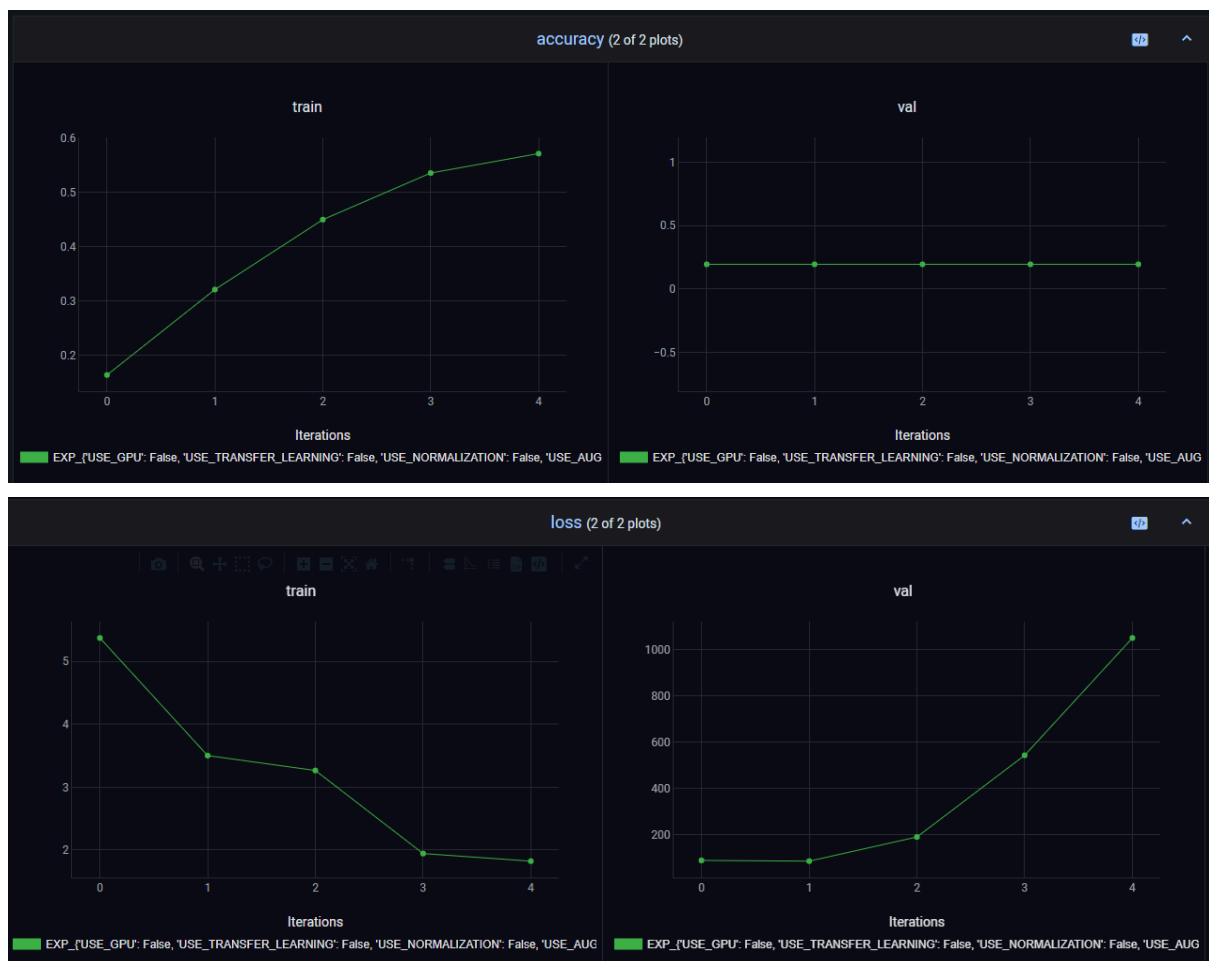
Owoce.zip zawiera klasy: Pomidory, Pomarańcze, Jabłka, Banany

Gruszka.zip jest folderem dodatkowych dla dodania danych.

2. Wytrenować model na sieci ResNet50 od zera (bez transfer learningu) na CPU  
- uzyskane wyniki (czas treningu oraz dokładność) będą stanowić base line dla kolejnych wyników.

**Wyniki: Accuracy: 0.1944, Loss: 1049.3859, Czas: 482.19 s**

Uzyskany wynik w 4 epochs





3. W celu optymalizacji szybkości treningu zastosować:

a. akcelerator GPU - przygotować raport porównawczy z czasem trenowania bez i z GPU,

Bez GPU

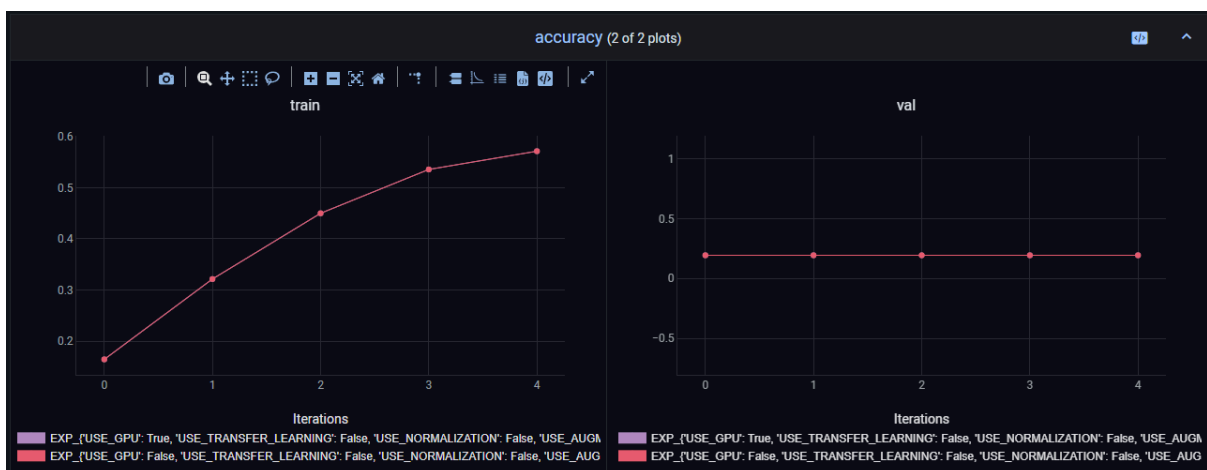
**Wyniki:** Accuracy:0.1944, Loss: 1049.3859, Czas: 475.11 s

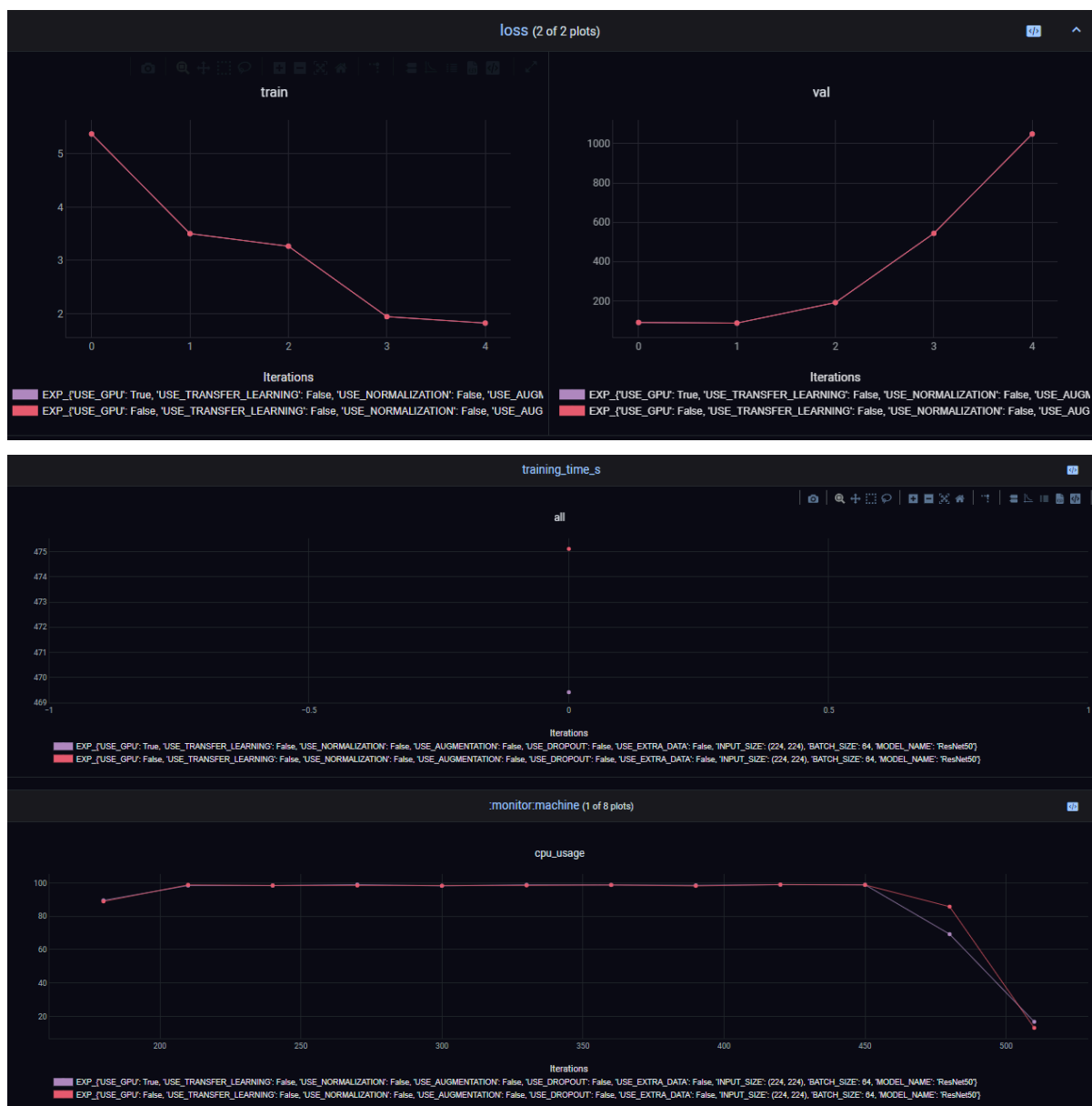
Uzyskany wynik w 4 epochs

Z GPU

**Wyniki:** Accuracy:0.1944, Loss: 1049.3859, Czas: 469.4154 s

Uzyskany wynik w 4 epochs



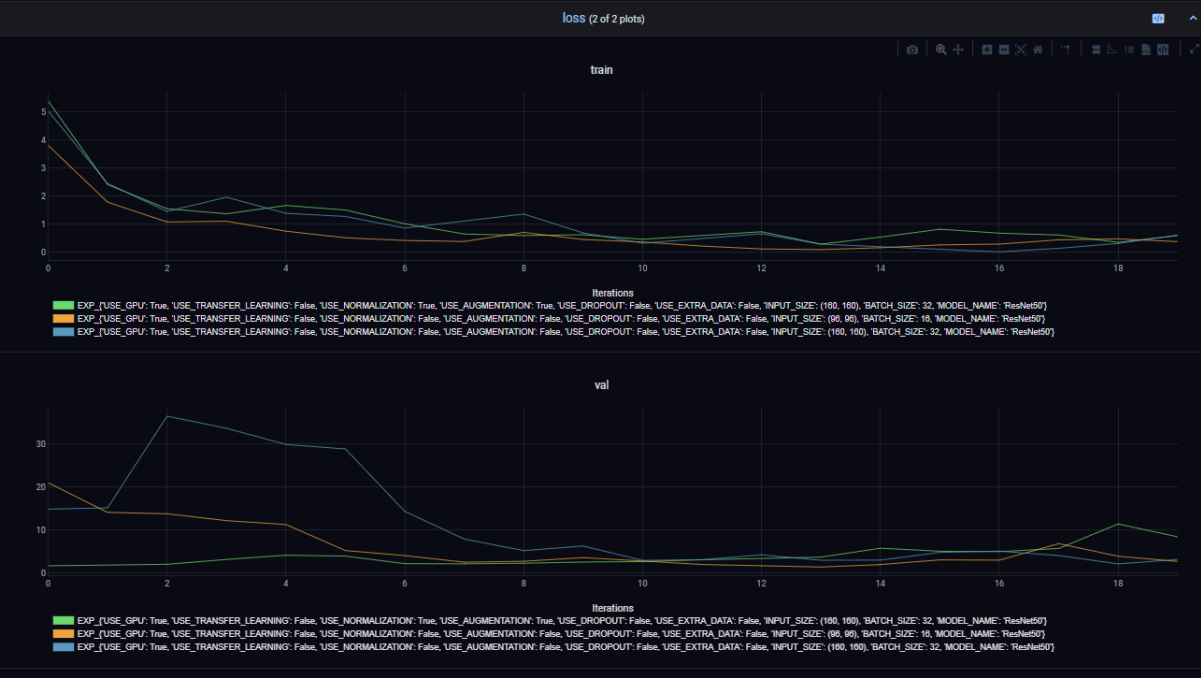
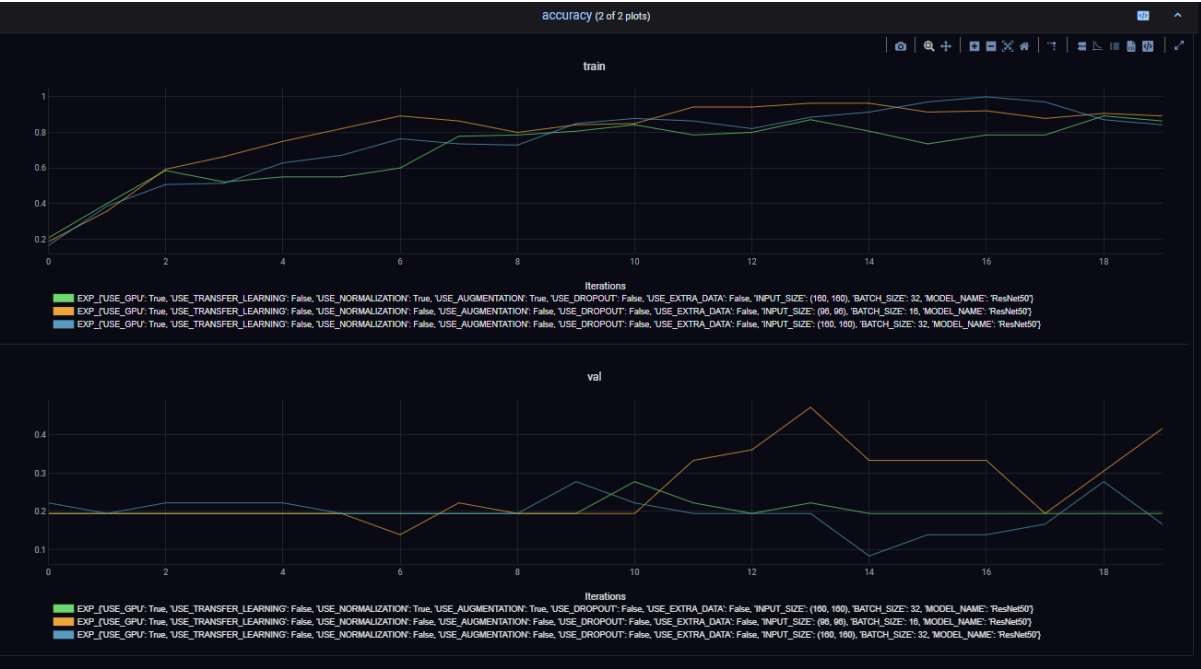


b. transfer learning - przygotować raport porównawczy z transfer learningiem i bez, interesuje nas tutaj dojście do pewnego poziomu dokładności np. 80%.

### Bez Transfer Learning

**Najlepsze Wyniki:** Accuracy: 0.4167, Loss: 2.3607, Czas: 458.94 s

Tutaj wykres odnosi się do 3 eksperymentów po 19 epochs dla uzyskania 80% accuracy (nie zostało uzyskane)



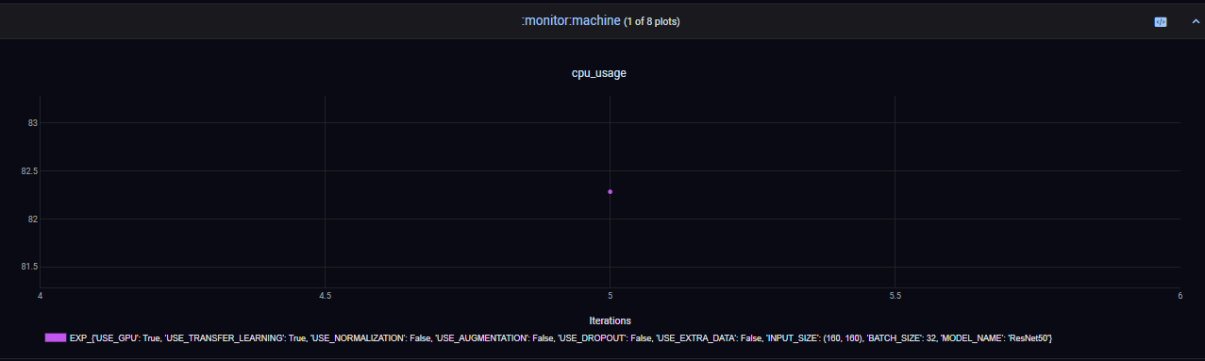
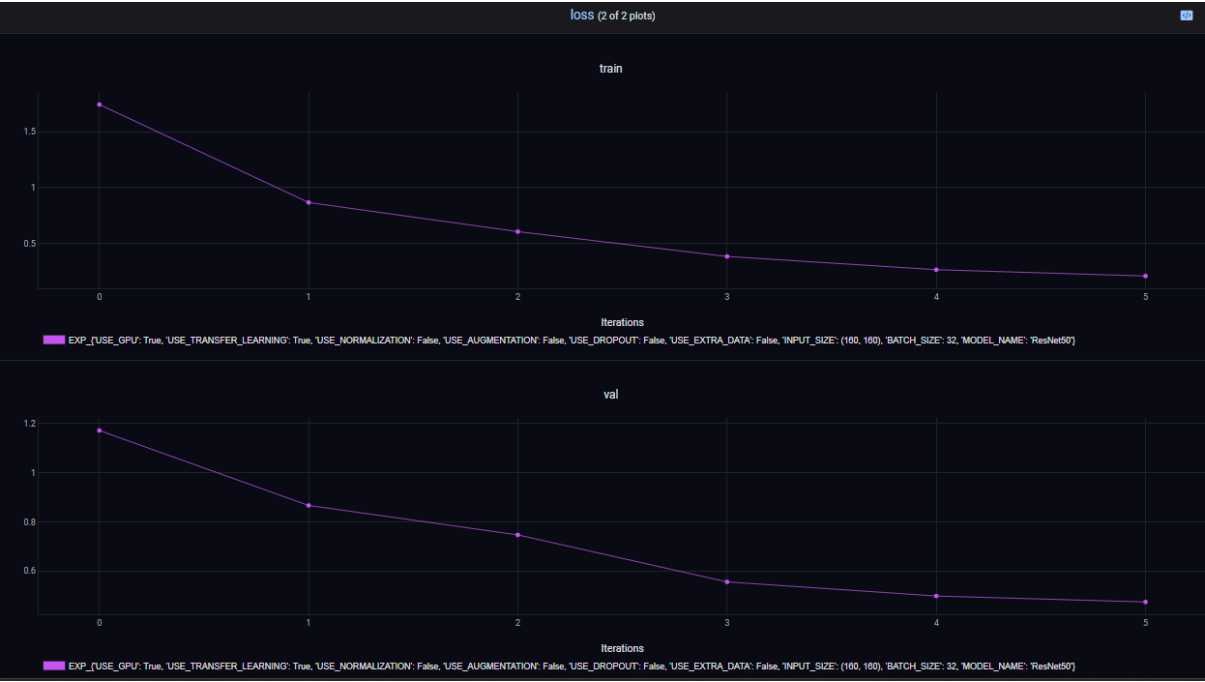


## Z Transfer Learning

**Wyniki: Accuracy: 0.8056, Loss: 0.4740, Czas: 77.061 s**

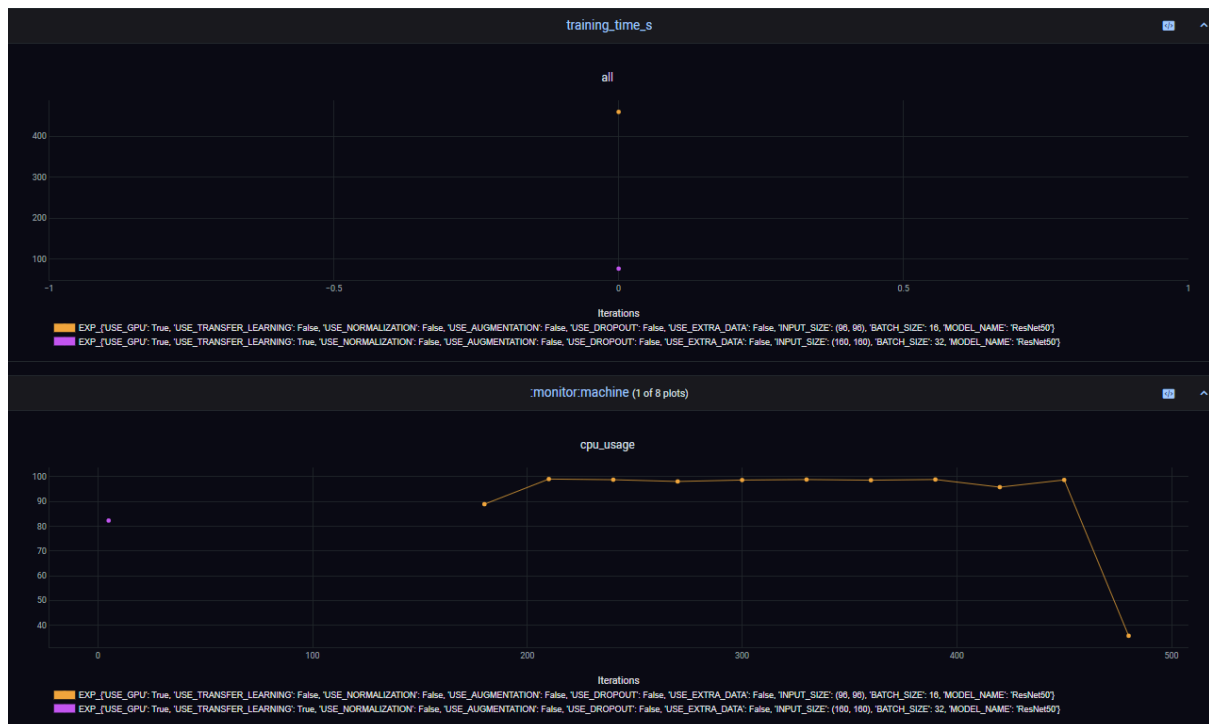
Uzyskany wynik w 5 epochs





Połączone wykresy porównawcze





4. W celu optymalizacji dokładności zastosować:

a. normalizację - przygotować raport porównawczy z normalizacją danych i bez.

Bez Normalizacji

**Wyniki:** Accuracy: 0.8056, Loss: 0.5197, Czas: 120.7198 s

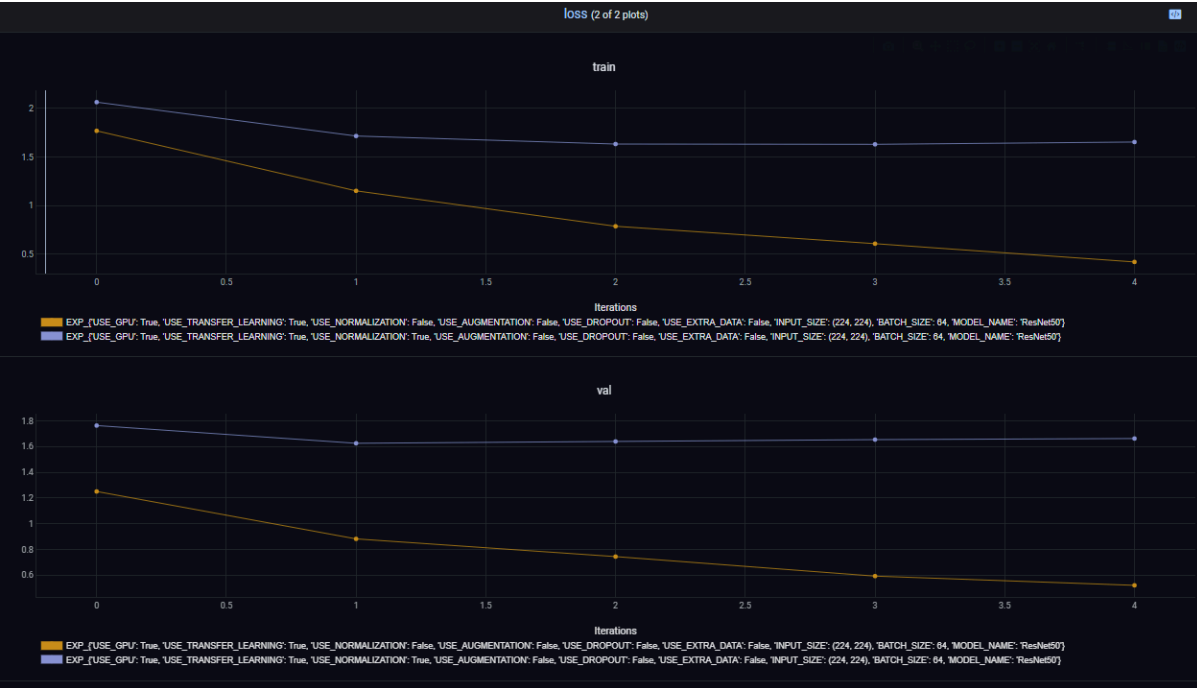
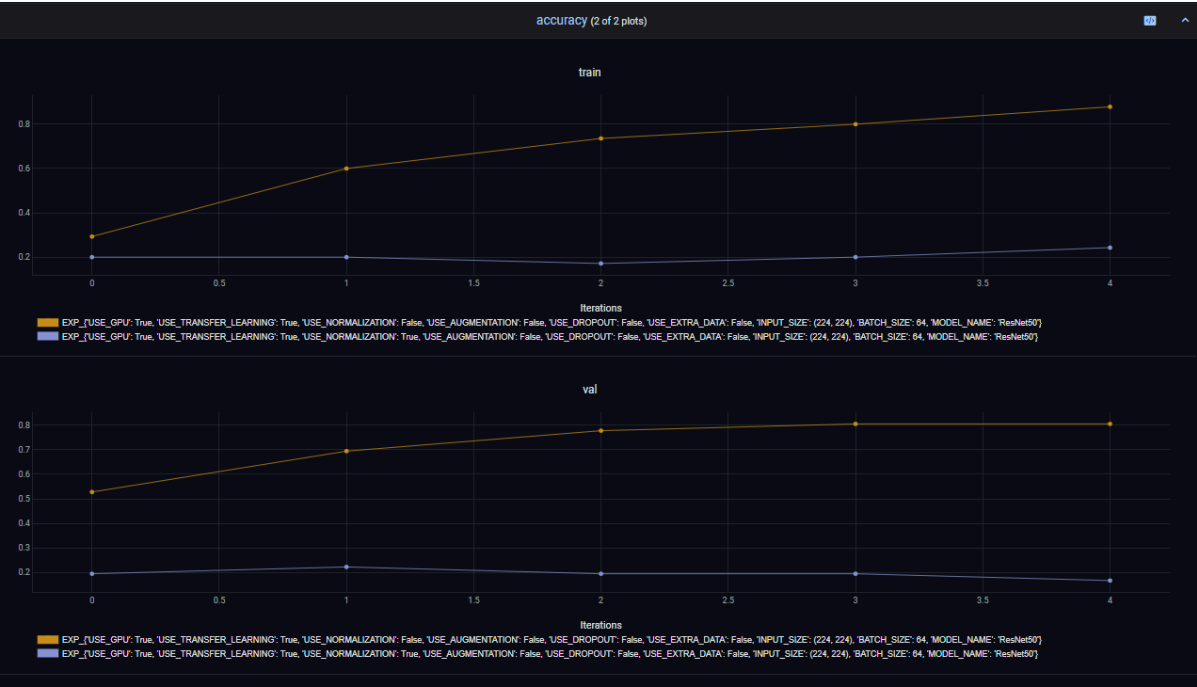
Uzyskany wynik w 4 epochs

Z Normalizacją

**Wyniki:** Accuracy: 0.1667, Loss: 1.6627, Czas: 137.0522 s

Uzyskany wynik w 4 epochs





b. augmentację danych - przygotować raport porównawczy z augmentacją i bez + informacją jakie przekształcenia zostały przeprowadzone.

### Bez Agumentacji

**Wyniki:** Accuracy: 0.1667, Loss: 1.6627, Czas: 137.7859 s

Uzyskany wynik w 4 epochs

### Z Agumentacją

**Wyniki:** Accuracy: 0.2222, Loss: 1.6765, Czas: 131.9355 s

Uzyskany wynik w 4 epochs





c. dropout - przygotować raport porównawczy z zastosowaniem dropoutu i bez.

### Bez Dropout

**Wyniki:** Accuracy: 0.2222, Loss: 1.6765, Czas: 145.1358 s

Uzyskany wynik w 4 epochs

### Z Dropout

**Wyniki:** Accuracy: 0.1944, Loss: 1.6249, Czas: 122.5361 s

Uzyskany wynik w 4 epochs





d. dokładanie danych - przygotować raport porównawczy z dołożeniem nowej partii danych i bez.

#### Bez Dodatkowych Danych

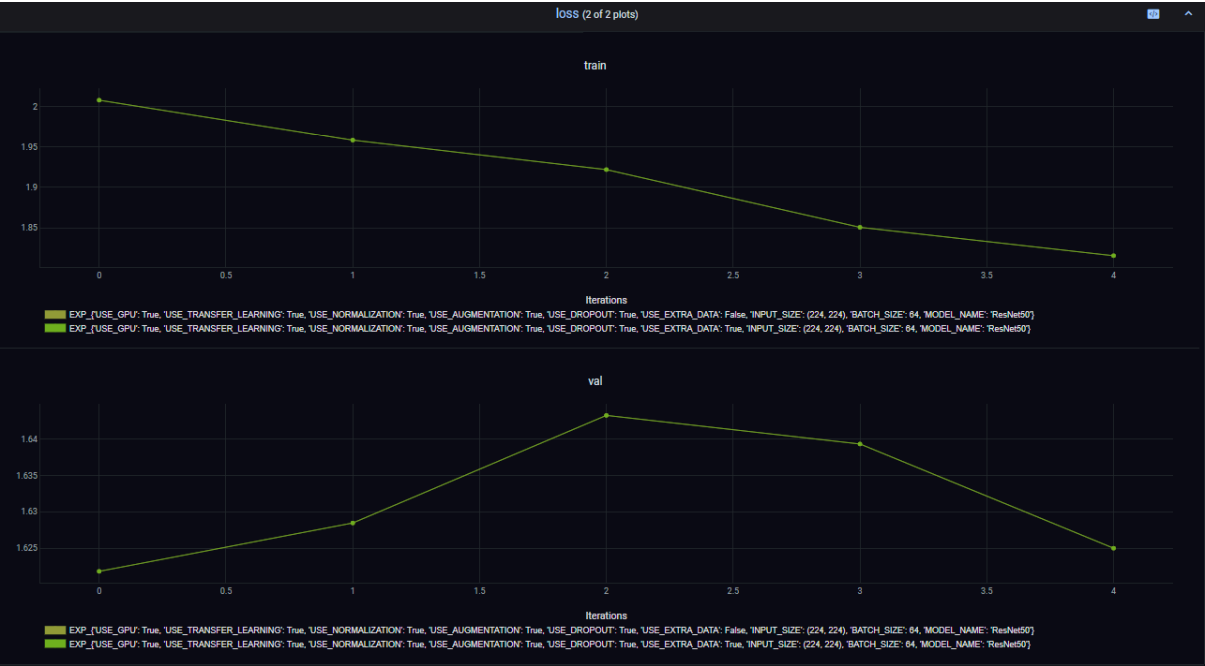
**Wyniki:** Accuracy: 0.1944, Loss: 1.6249, Czas: 160.7213 s

Uzyskany wynik w 4 epochs

#### Z Dodatkowymi Danymi

**Wyniki:** Accuracy: 0.1944, Loss: 1.6249, Czas: 128.0419 s

Uzyskany wynik w 4 epochs





e. różne rozmiary wejściowe (np. 96x96, 160x160, 224x224) - przygotować raport porównawczy dla każdego rozmiaru.

96x96

**Wyniki:** Accuracy: 0.1944, Loss: 1.6161, Czas: 31.69 s

Uzyskany wynik w 4 epochs

160x160

**Wyniki:** Accuracy: 0.2222, Loss: 1.6227, Czas: 68.20 s

Uzyskany wynik w 4 epochs

224x224

**Wyniki:** Accuracy: 0.1944, Loss: 1.6250, Czas: 143.17 s

Uzyskany wynik w 4 epochs



f. różny rozmiar Batch size (np. 32, 64, 128) - przygotować raport

porównawczy dla każdego rozmiaru.

32

**Wyniki:** Accuracy: 0.1111, Loss: 1.6152, Czas: 138.09 s

Uzyskany wynik w 4 epochs

64

**Wyniki:** Accuracy: 0.1944, Loss: 1.6250, Czas: 144.67 s

Uzyskany wynik w 4 epochs

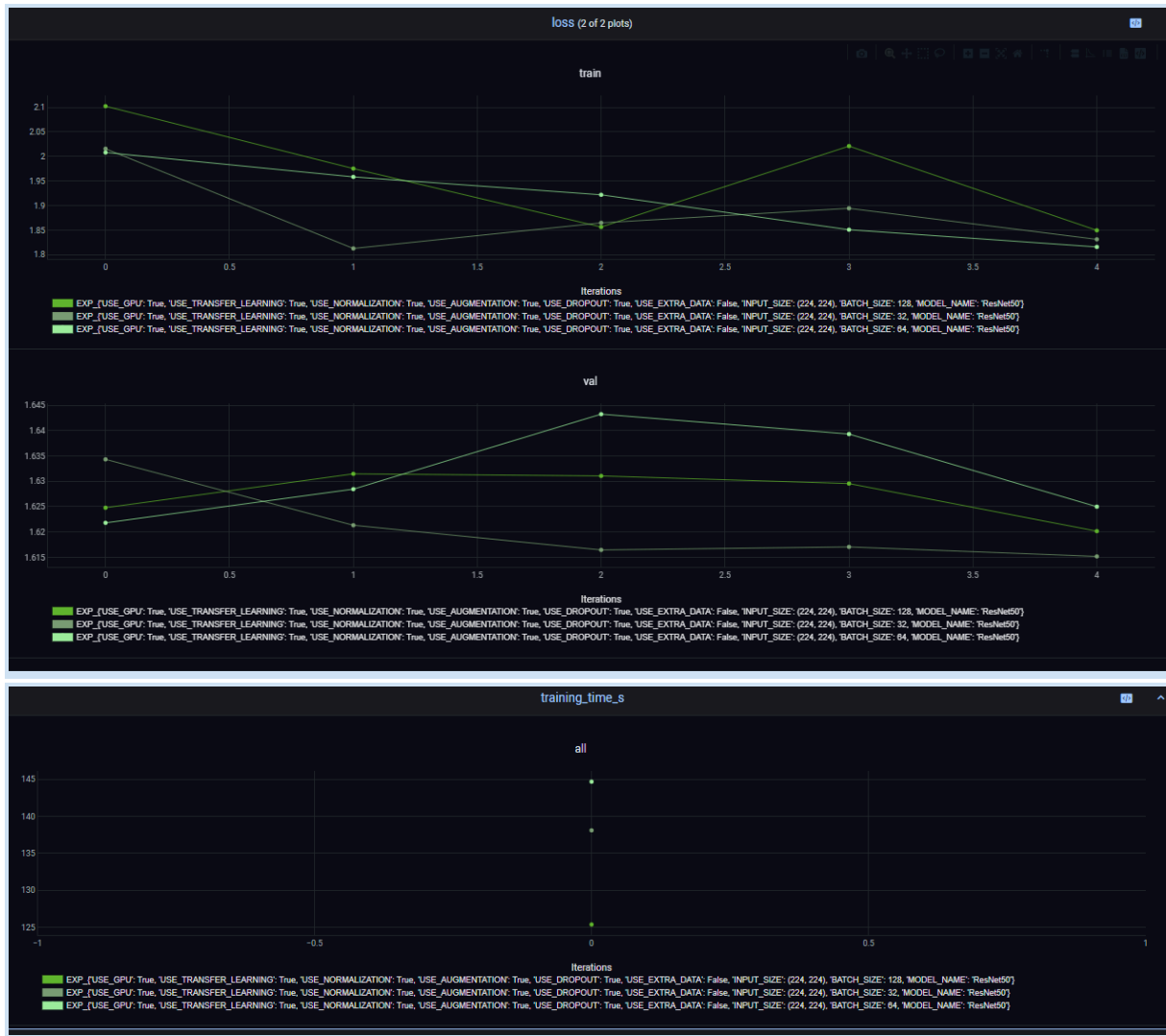
128

**Wyniki:** Accuracy: 0.1667, Loss: 1.6202, Czas: 125.36 s

Uzyskany wynik w 4 epochs







g. różne struktury sieci (np. VGG16, ResNet101, InceptionV3, MobileNet) - przygotować raport porównawczy dla każdej sieci (min. 4).

Parametry True

VGG16

**Wyniki:** Accuracy: 0.2778, Loss: 1.5482, Czas: 372.4804 s

Uzyskany wynik w 4 epochs

ResNet101

**Wyniki:** Accuracy: 0.25, Loss: 1.5960, Czas: 217.9898 s

Uzyskany wynik w 4 epochs

InceptionV3

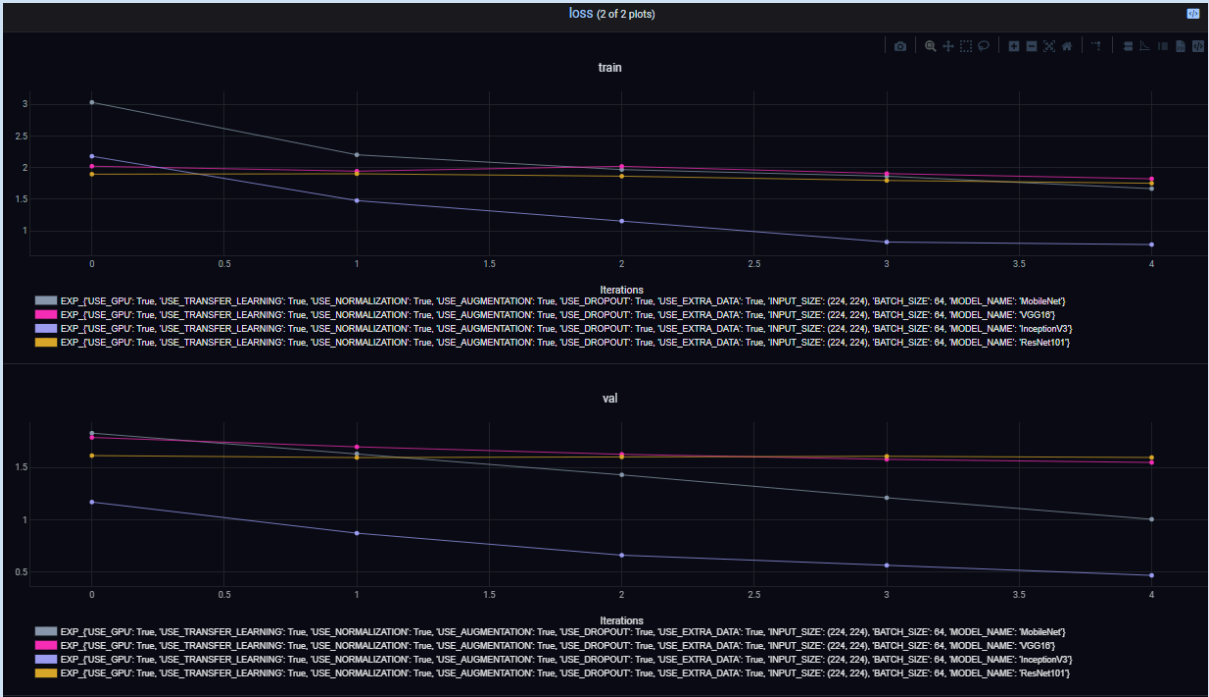
**Wyniki:**Accuracy: 0.8333, Loss: 0.46698, Czas: 91.0564 s

Uzyskany wynik w 4 epochs

MobileNet

**Wyniki:**Accuracy: 0.6666, Loss: 1.0041, Czas: 39.1148 s

Uzyskany wynik w 4 epochs



Parametry False

VGG16

**Wyniki:** Accuracy: 0.4167, Loss: 4.2569, Czas: 372.42 s

Uzyskany wynik w 4 epochs

ResNet101

**Wyniki:** Accuracy: 0.7778, Loss: 0.5474, Czas: 216.01 s

Uzyskany wynik w 4 epochs

InceptionV3

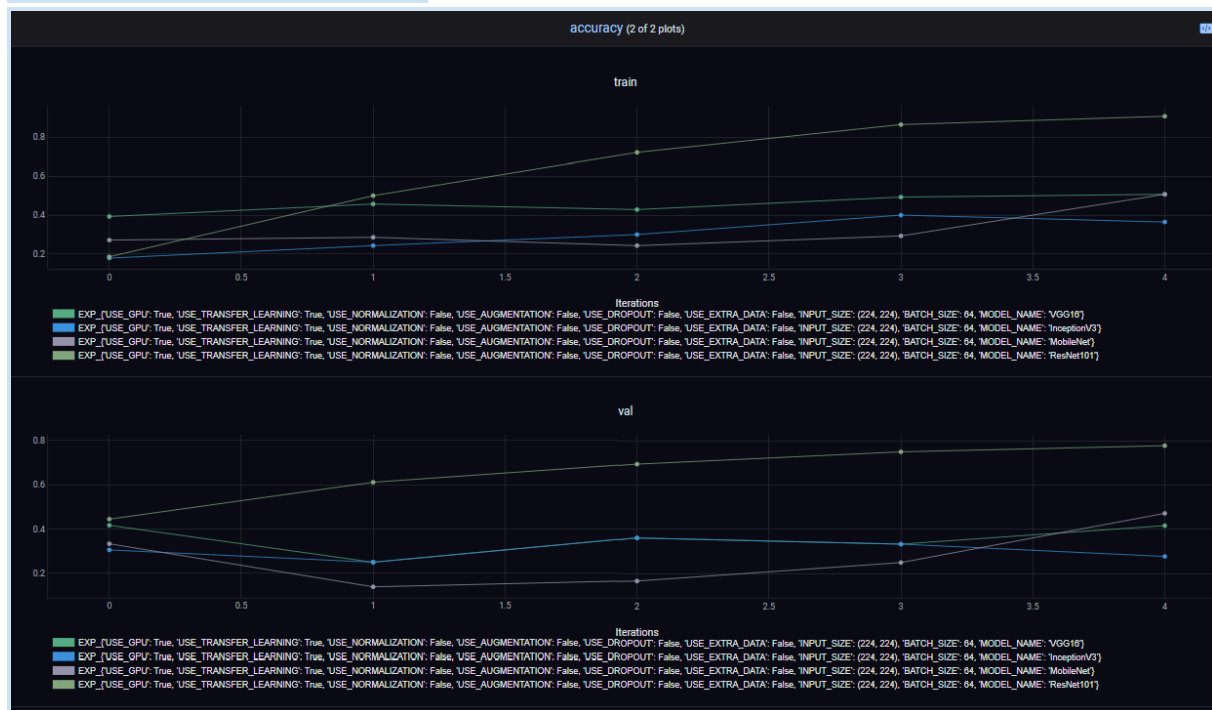
**Wyniki:** Accuracy: 0.2778, Loss: 8.8007, Czas: 90.27 s

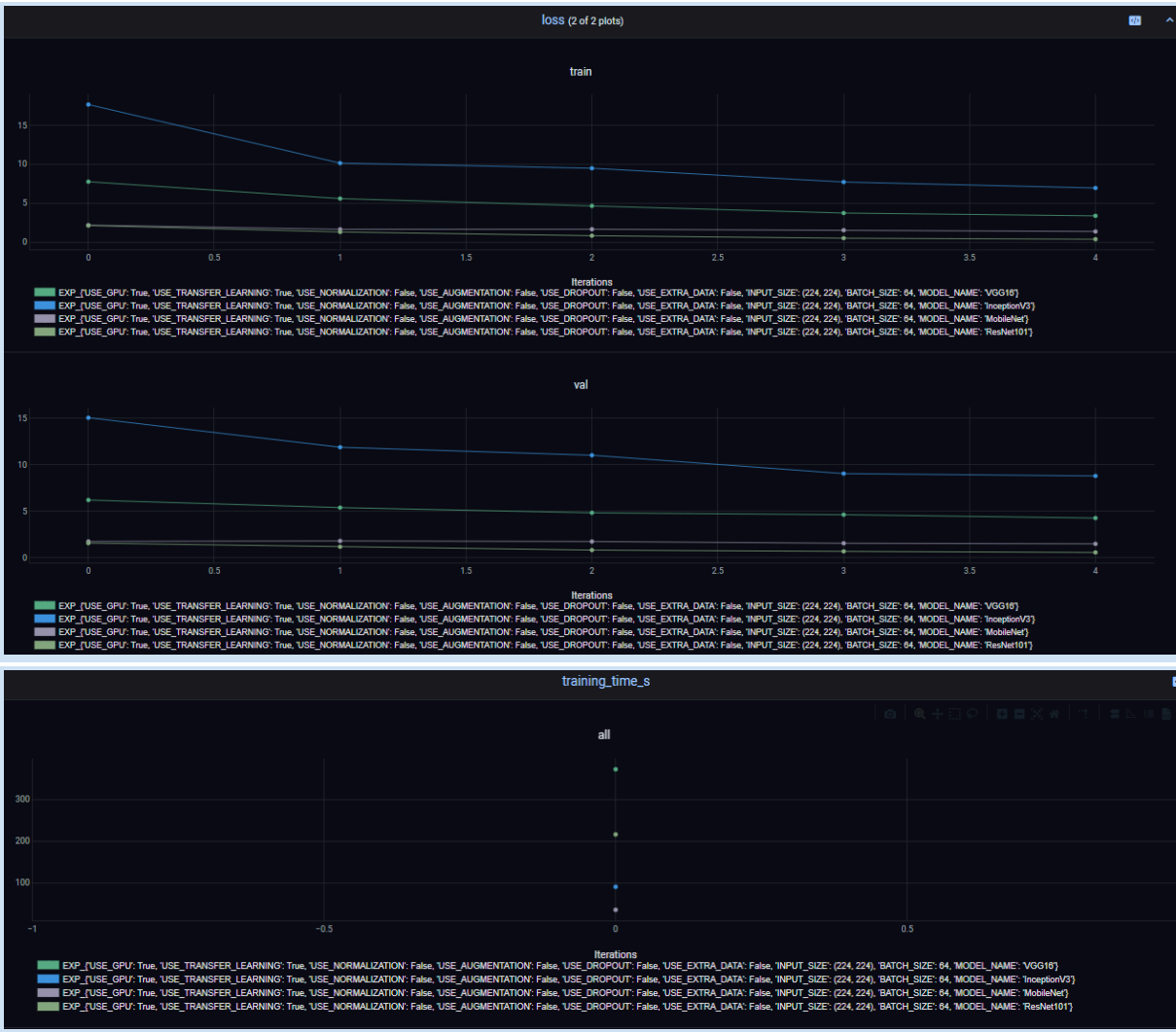
Uzyskany wynik w 4 epochs

MobileNet

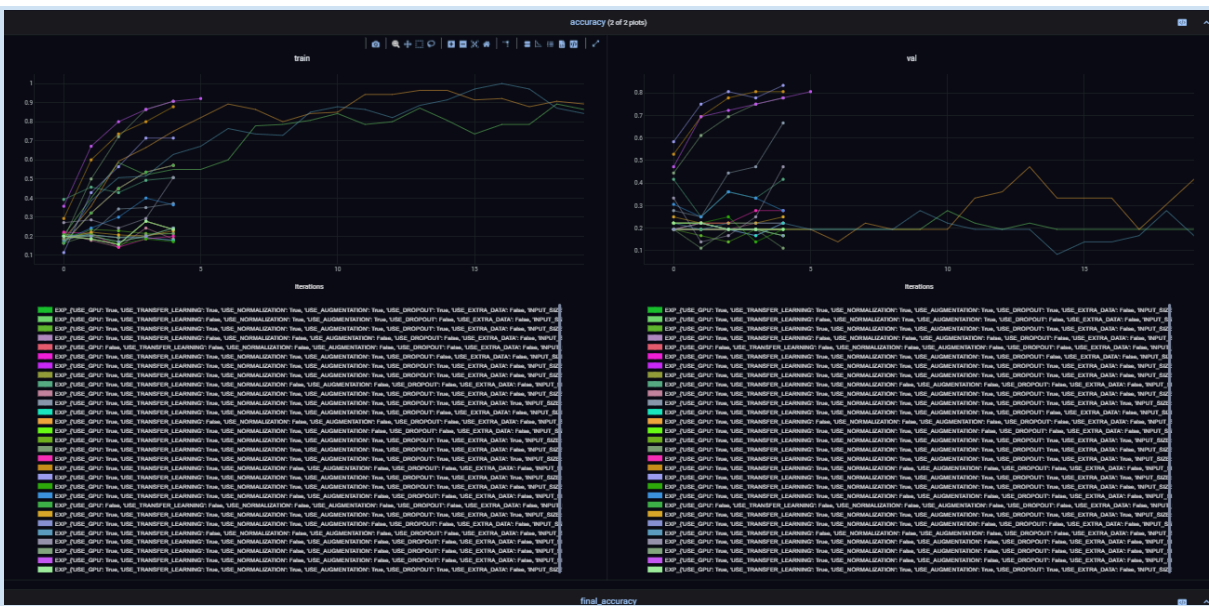
**Wyniki:** Accuracy: 0.4722, Loss: 1.4727, Czas: 35.00 s

Uzyskany wynik w 4 epochs





Wykres wszystkich testów:



Wnioski:

Transfer learning ma największy wpływ na poprawę dokładności i czasu treningu.

Dobór architektury i parametrów wejściowych (rozmiar obrazu, batch size) znacząco wpływa na efektywność modelu.

Nie wszystkie parametry (np. normalizacja, dropout) poprawiają wynik - wszystko zależy od kontekstu modelu.